

Projet Airlines

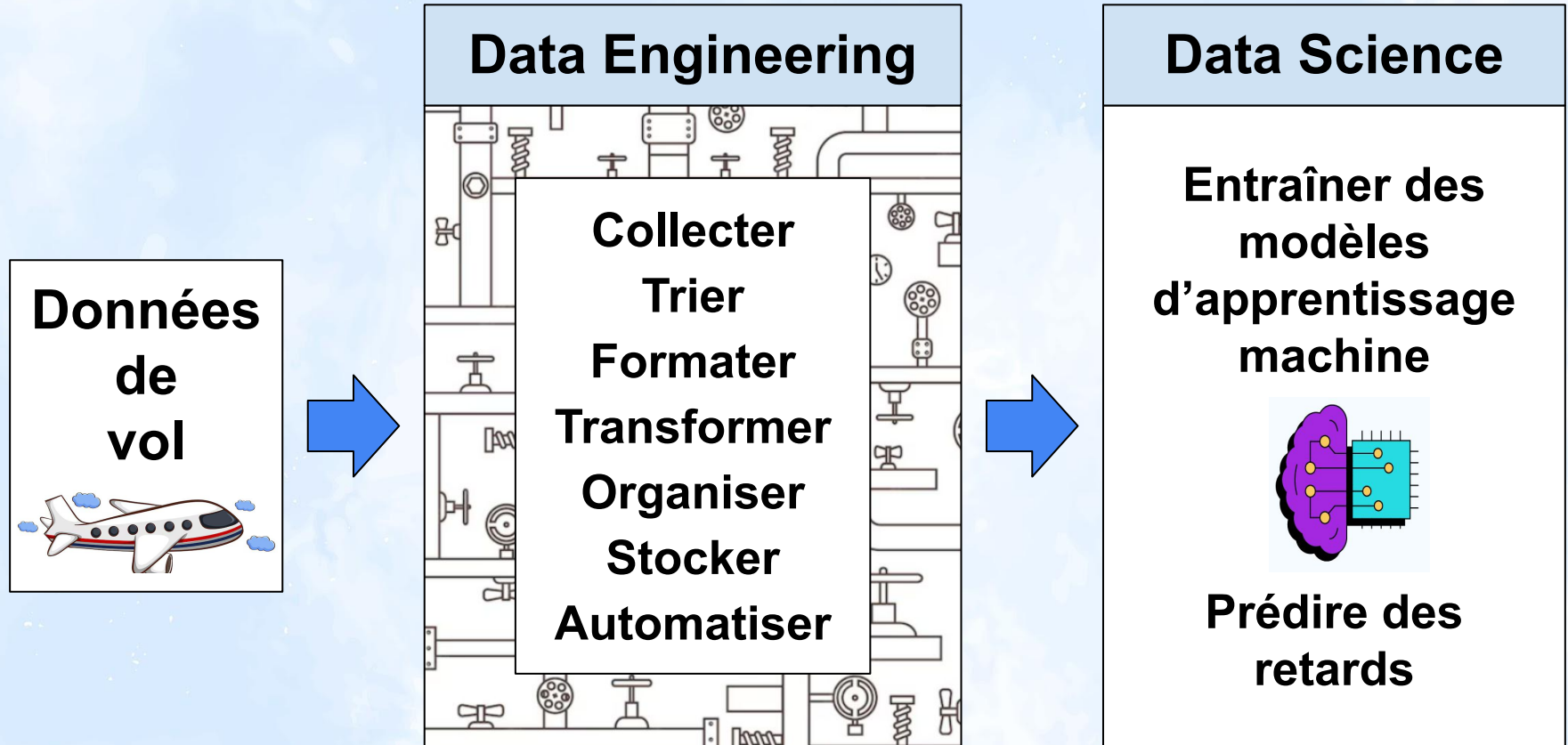
Nicolas CALO, Johan CLOOS, Younes ES-SOUALHI,
Rathana LAT, Youssef ZNATI

Data Engineer
Bootcamp DST septembre 2025

Plan de la présentation

1. Collecte des données
2. Stockage et tri des données - mongoDB
3. Base de données relationnelle - PostgreSQL
4. Exploration des données
5. Apprentissage machine
6. Conteneurisation, APIs et dashboard
7. Déploiement sur le cloud

Objectifs du projet



1 - Collecte de Données

Sources de données retenues

Source	Type de données	Utilité
API 	Informations sur les vols Air France KLM	Vols passés pour apprentissage machine Vols futurs pour prédictions
Wikipedia 	Détails géographiques des aéroports mondiaux	Enrichissement du jeu de données avec continent, région, etc.
Github IP2Location 	Correspondance code IATA/ICAO des aéroports	Correspondance données Wikipédia et Air France KLM

Collecte des données

API Air France KLM - Description de l'API

Méthode de requête	requêtes GET avec paramètres
Authentification nécessaire	clé API
Format de réponse	.json
Limites	- 100 requêtes par jour - max 100 vols par requêtes - de 6 mois dans le passé à 1 an dans le futur

Script Python à déclenchement journalier
librairies “pandas” et “requests”

Collecte des données

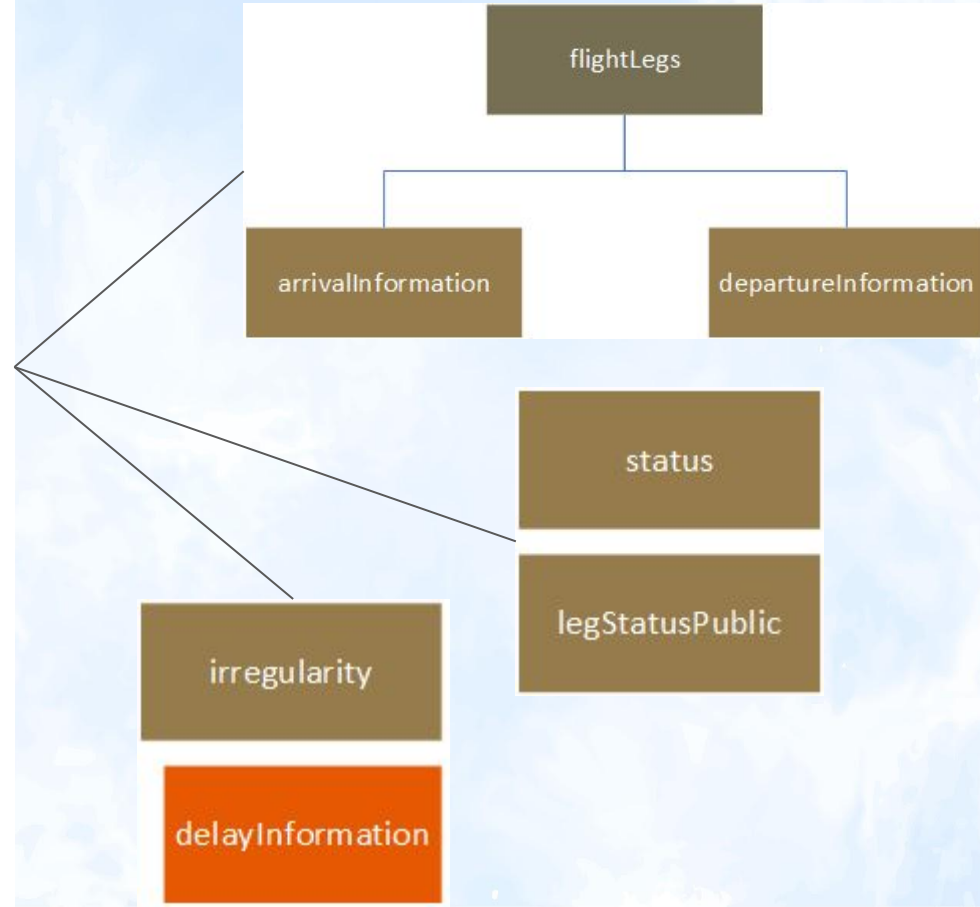
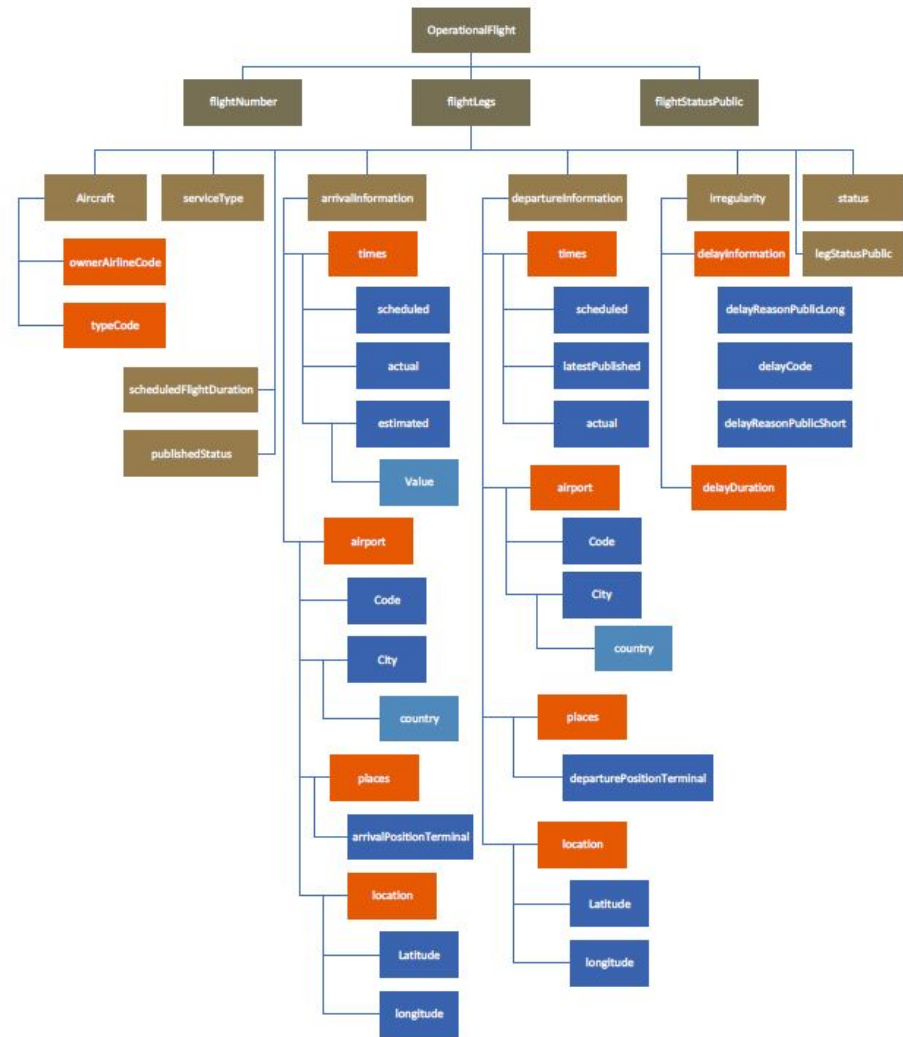
API Air France KLM - Stratégie de collecte

Contrainte technique	Solution
Limites de requêtes	- Roulement des clés API - 30 plus gros aéroports européens seulement
Évitement des doublons	- Comparaison entre requête et fichiers déjà récupérés
Actualisation des vols prévus	- Téléchargement / Actualisation chaque jour des vols
Taille des données	- Compression / Suppression des fichiers

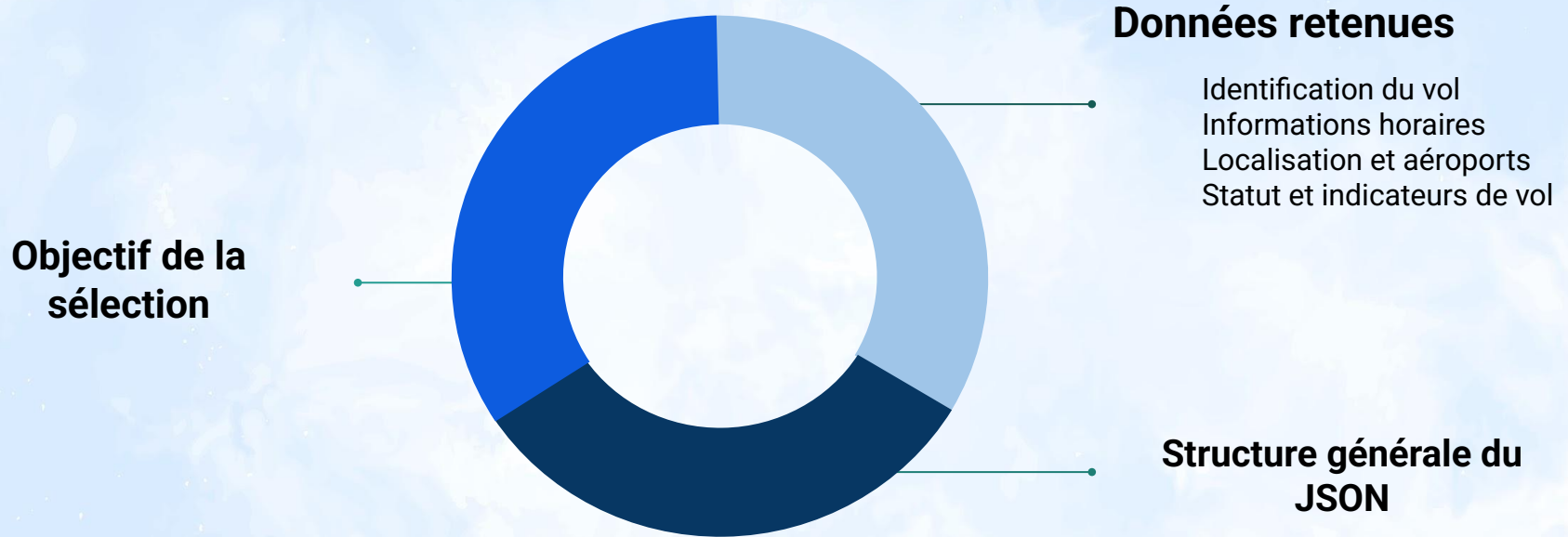
> 1'000'000 de vols historiques et 240'000 vols planifiés

2 - Stockage et tri des données mongoDB

Arborescence JSON



Données pertinentes



Base de données NoSQL

- Objectif
 - Stocker et manipuler des données semi-structurées
 - Objet sous format json (Orienté document)
- Choix :



Insertion data



{.json}



Fichiers compressés

Operational flights

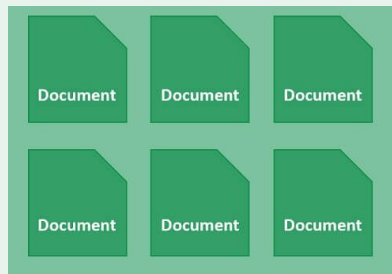
mongoDB

Les collections

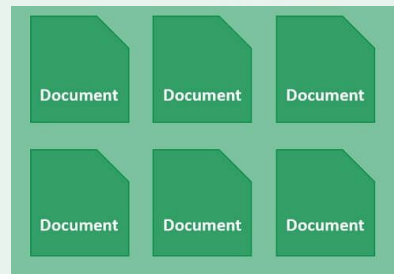
MongoDB



Scheduled
*(Vols programmés disponibles
de jusqu'à un an)*



Scheduled J-1
(mis à jour 24 h avant le départ)

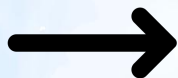


Historique
(Depuis Avril 2025)

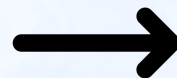
Export des données en format tabulaire



mongoDB



Interface utilisateur



Fichiers .csv pour
importation SQL

3 - Base de données relationnelle

Base de données relationnelle

- Objectif
 - Implémenter un SGBDR
 - Alimenter la base de donnée
 - Avoir des données structurées

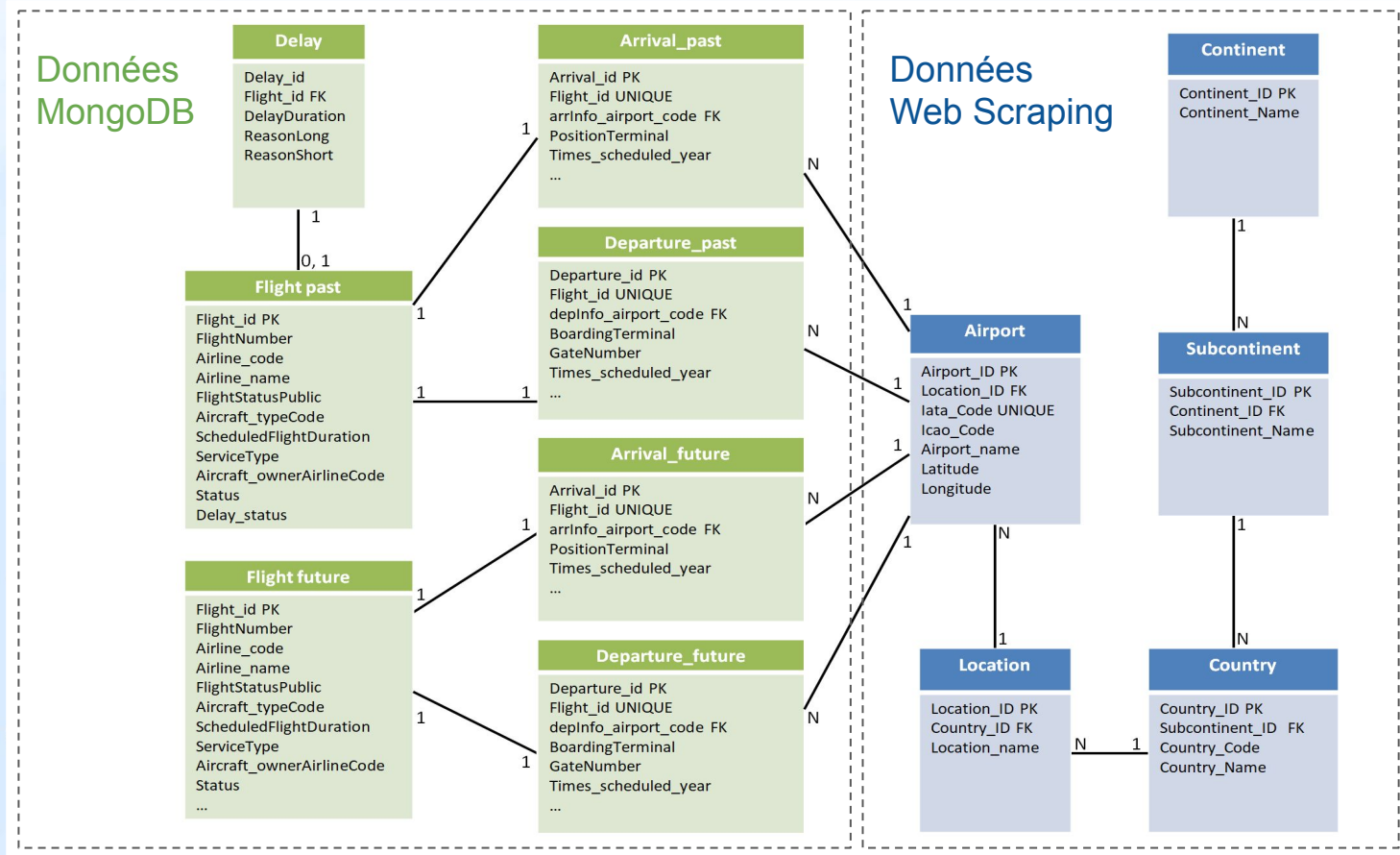
- Choix :



Base de données relationnelle

Données statiques	Données quotidiennes
● Source Wikipédia ○ Géographique ○ Aéroportuaire ● Extraction par web scraping	● Source MongoDB ○ Information des vols ● Importation de fichiers csv

Base de données relationnelle



Base de données relationnelle

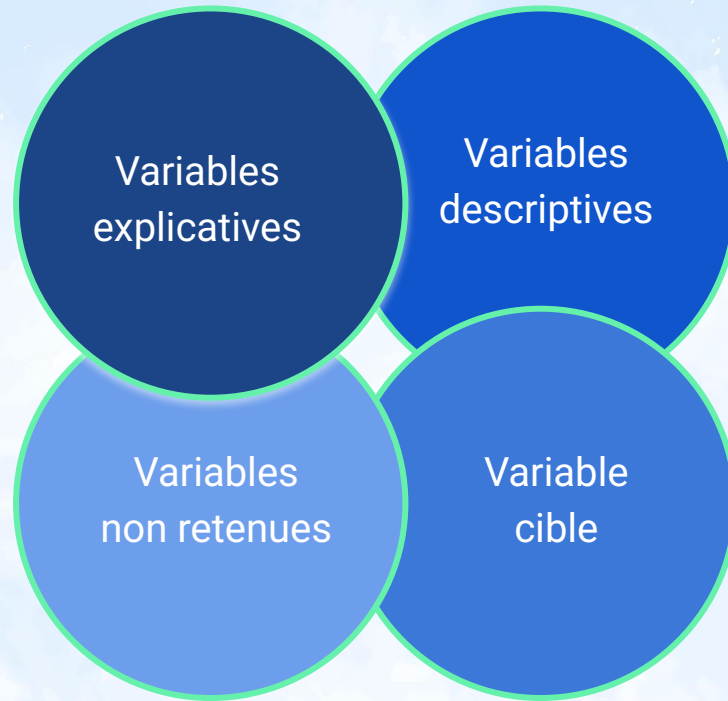
- Problèmes rencontrés et résolus
 - Gestion des doublons
 - Adaptation en continu des scripts en fonction du besoin du projet
 - Correspondance sur les codes IATA entre MongoDB et le web scraping
 - Performance

Base de données relationnelle

- Résultat
 - Base de données opérationnelle
 - Données consolidées et structurées :
 - du continent à la porte d'embarquement
 - Données disponibles pour :
 - Machine Learning (passées)
 - Interface utilisateur Dash (futures)

4 - Exploration des données

Etude des données tabulaires



Préparation des données au ML



Nettoyage du jeu de données

Analyse de la variable flightStatusPublic

flightStatusPublic	Taux (%)	
ARRIVED	91,2516	✓
ON_TIME	5,5515	✓
CANCELLED	3,0631	✓
PARTIALLY_CANCELLED	0,1323	✗
DELAYED_DEPARTURE	0,0013	✗
DEPARTED	0,0001	✗
NEW_DEPARTURE_TIME	0,0001	✗

Analyse de la variable flightLegs_serviceType

Type	Correspondance du type	Taux	
J	Normal Service (vol programmé)	98,6143	✓
U	Service operated by Surface Vehicle	0,7257	✗
C	Passenger Only	0,3500	✗
G	Normal Service (vol non planifié)	0,2948	✗
S	Shuttle Mode	0,0121	✗
O	Migrants/ Immigrants	0,0025	✗
L	Passenger and Cargo/ Mail	0,0005	✗

5 - Consommation de la donnée Apprentissage machine

Apprentissage machine

Pipeline SciKit-Learn



Valeurs explicatives	type d'avion, saison, aéroport de départ, aéroport d'arrivée, période de départ, période d'arrivée, weekend, durée du vol		
Problème	Classification		Régression
	Status du vol	Gamme de retard	Durée du retard
	ONTIME LATE CANCELLED	0-5 min 5-15 min 15-30 min...	0 - 360 min
Algorithmes testés	Logistic_OVO (One vs One) Logistic_OVR (One vs Rest) DecisionTree RandomForest		LinearRegression XGBRegressor DecisionTreeRegressor RandomForestRegressor

+ **Hyperparameter tuning**

Apprentissage machine

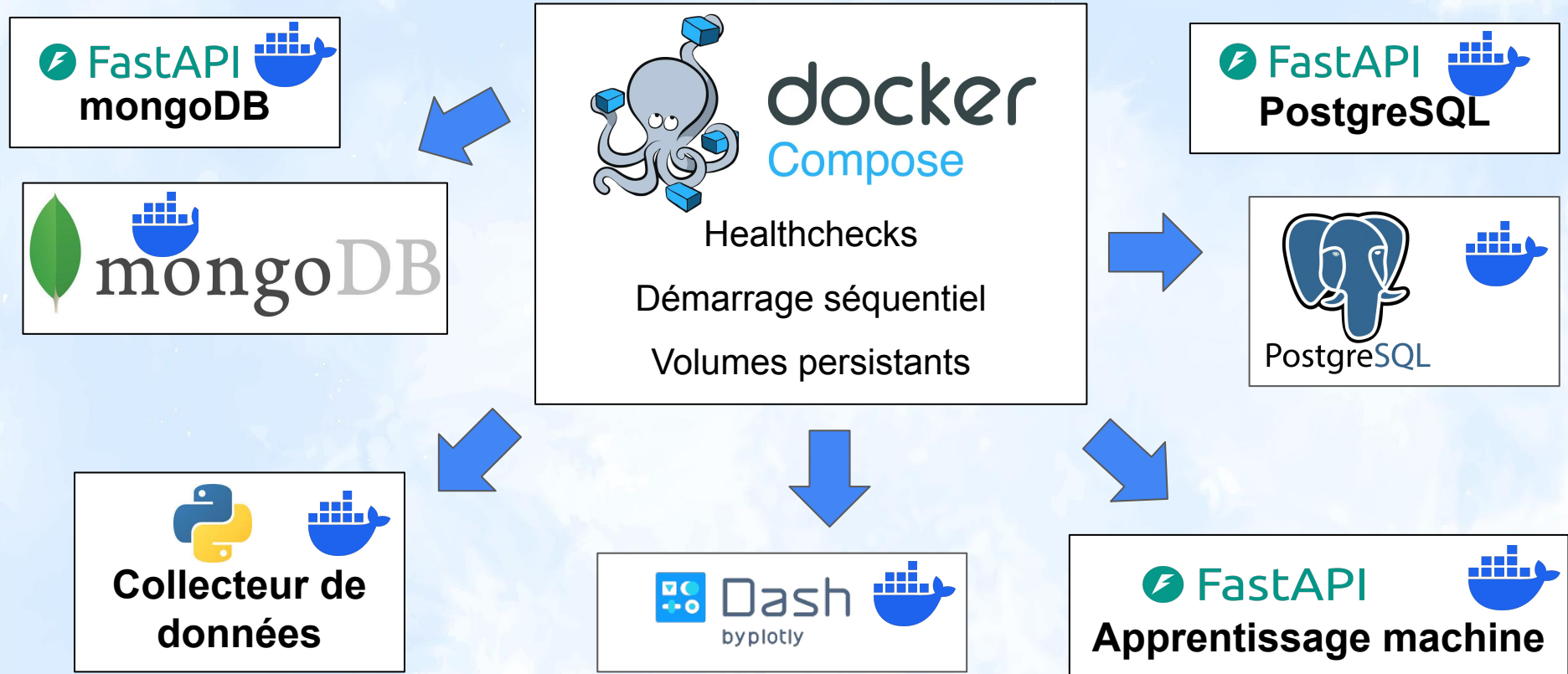
Meilleurs modèles



Status	Retard	
> 450'000 (80%) vols train > 100'000 (20%) vols test	> 170'000 (80%) vols train > 40'000 (20%) vols test	
Classification		Régression
Status du vol	Gamme de retard	Durée du retard
ONTIME LATE CANCELLED	0-5 min 5-15 min 15-30 min...	0 - 360 min
DecisionTree Précision : 0.874	Logistic_OVR Précision : 0.335	XGBRegressor r^2: 0.068
Plutôt bien	Mauvais	Peu utilisable en l'état

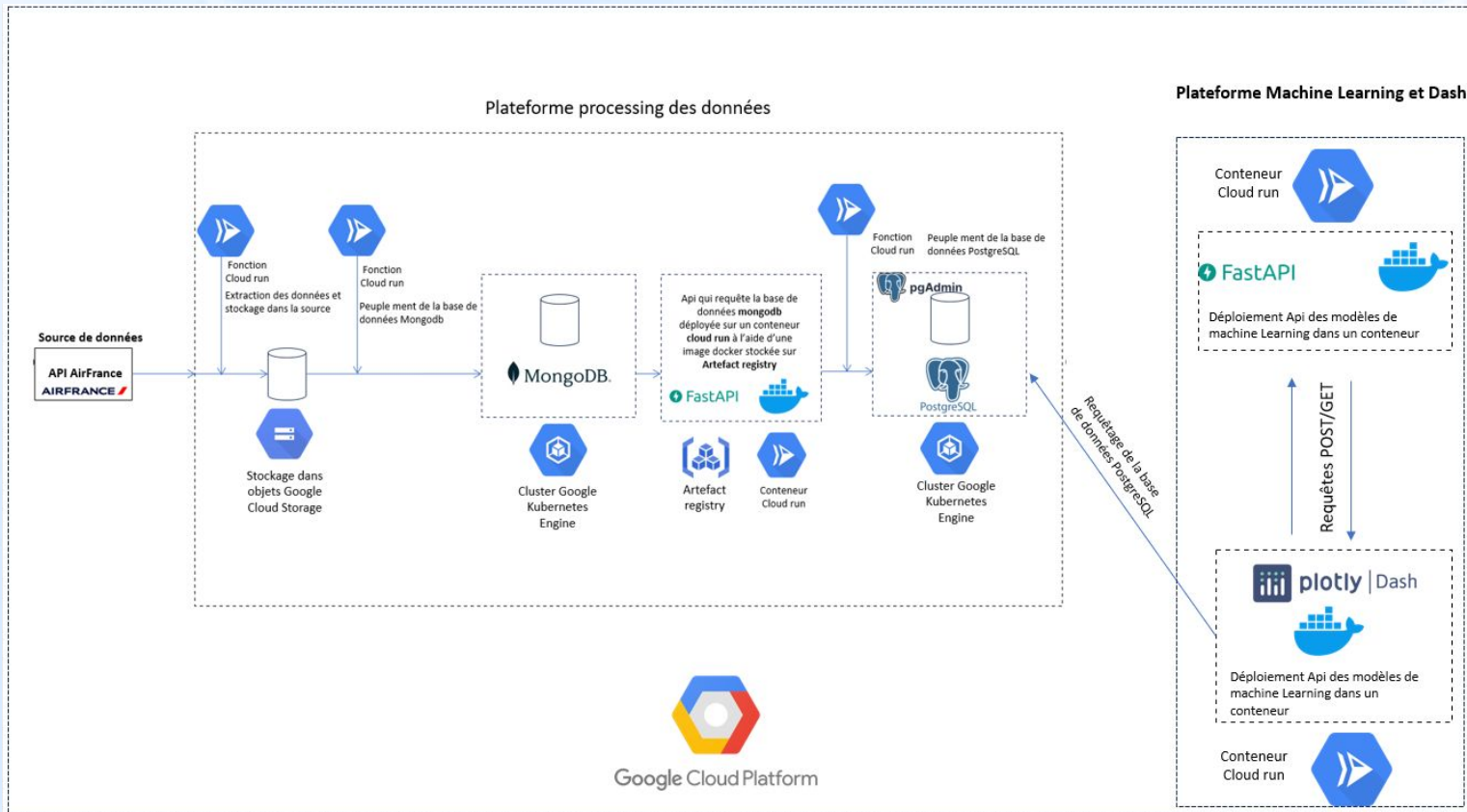
6 - Conteneurisation, APIs et dashboards

Déploiement en local avec Docker



7 - Déploiement sur le cloud

Workflow actuellement déployé sur gcp



Déploiement d'API sur GCP

Récupération du
répertoire contenant
le dockerfile



Dockerfile

Déploiement de
l'image
sur un conteneur
cloud run



Cloud Build



Artefact registry

Construction de
l'image
au sein d'artefact
registry
avec Cloud Build



Cloud run



Google Cloud Platform

Déploiement des fonctions sur GCP



Cloud
Scheduler



Pub/Sub



Cloud Run
Function

Configuration du scheduler pour envoyer le message au broker pubsub sur un interval de temps

réception du message dans le topic

déclenchement la fonction après réception du message

Exemple déploiement d'une fonction

Cloud Functions



☐	Nom ↑	Type de déploiement
☐	clean-populate-fact-tables-in-postgre	(--) Fonction
☐	data-api	📁 Dépôt
☐	dst-de-airlines-api	(🔗) Conteneur
☐	extract-gcs-to-mongo-db	(--) Fonction
☐	new-extract-function	(--) Fonction
☐	populating-mongodb-future-postgre	(--) Fonction
☐	populating-mongodb-future-postgre-d1	(--) Fonction
☐	populating-mongodb-past-postgre	(--) Fonction

[← Informations sur le service](#) [✎ Modifier et déployer la nouvelle révision](#) [🔗 Se connecter au dépôt](#) [🧪 Test](#) [📖 Apprendre](#) [🔄 Actualiser](#)

populating-mongodb-past-postgre Région : europe-west9 URL : <https://populating-mongodb-past-postgre-901450845940.europe-west9.run.app> Scaling : automatique (minimum : 1, maximum : 5) ✎

Observabilité Révisions **Source** Déclencheurs Réseau Sécurité YAML

Source Image de base : Python 3.12 (Ubuntu 22) Point d'entrée de la fonction : populate_mongodb_past_in_postgre [Modifier la source](#) [Afficher la charge utile](#)

main.py

requirements.txt

```
1 import base64
2 import functions_framework
3 from workflow_mongodb_postgresql_functions.utilities import get_dataframe_from_mongodb, load_postgres_config, copy_dataframe_to_postgres, run_sql_from_string
4 from workflow_mongodb_postgresql_functions.gcp_logger import logger
5 import json
6 from io import BytesIO
7 from google.cloud import storage
8 import os
9 from dotenv import load_dotenv
10 load_dotenv()
11
12
13
14 # Triggered from a message on a Cloud Pub/Sub topic.
15 @functions_framework.cloud_event
16 def populate_mongodb_past_in_postgre(cloud_event):
17     load_dotenv()
18     # loading inputs
19     collection_name, table_name, postgres_db_config = load_postgres_config()
20     BUCKET_NAME = os.getenv("BUCKET_NAME")
21     SOURCE_BLOB_NAME = os.getenv("MAPPING_FILE_BLOB_NAME")
22     logger.info(f"Fetching column mapping JSON from GCS bucket '{BUCKET_NAME}', blob '{SOURCE_BLOB_NAME}' ...")
23
24     storage_client = storage.Client()
25     bucket = storage_client.bucket(BUCKET_NAME)
26     blob = bucket.blob(SOURCE_BLOB_NAME)
27
28
29     json_data = blob.download_as_bytes()
30     column_mapping = json.loads(BytesIO(json_data))
31     logger.info(f"Column mapping loaded with {len(column_mapping)} entries.")
32
33     df = get_dataframe_from_mongodb(collection=collection_name)
34     # import pandas as pd
35     # df = pd.read_csv("workflow_mongodb_postgresql_package/afkm_flight_from_mongo_filtered_20251113-21-36-51.csv.gz")
36
37     # Rename DataFrame columns according to the mapping
38     logger.info("Renaming DataFrame columns...")
39     df.rename(columns=column_mapping, inplace=True)
40     logger.info("Columns renamed successfully.")
```

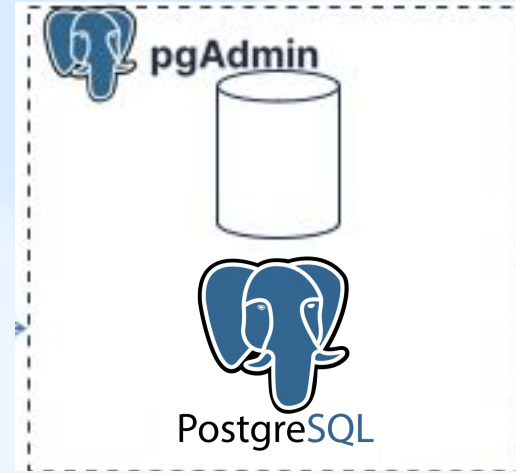
[Télécharger l'archive](#)

Déploiement de mongoDB et PostgreSQL sur GKE



Fichiers kubernetes pour le déploiement :

- Secret :
 - déploiement des identifiants en mode opaque .
- StatefulSet :
 - pull de l'image
 - définition de l'env (identifiant, montage volumes...)
 - définition de la demande en ressource (RAM, CPU),
 - définition du volume persistant
- Service :
 - définition de l'exposition de la base de données.



mongoDB et postgresQL ont été déployés sur deux cluster kubernetes indépendants

Problèmes rencontrés lors du déploiement

Problèmes :

- Erreurs de Time out : liées à la durée d'exécution des scripts.
- Limitation des ressources en raison du budget d'essai.
- Requêtes gourmandes pour les transformations entre mongoDB et PostgreSQL.

Solutions :

- Optimisation des fonctions et des boucles dans les scripts.
- Traitement par batch les différents documents qui proviennent de mongoDB.

Cloud Monitoring

Journaux des logs

[à page précédente](#)



populating-mongodb-future-postgre-d1

Région : europe-west9

URL : <https://populating-mongodb-future-postgre-d1-901450845940.europe-west9.run.app>



Scaling : automatique (minimum : 1, maximum : 5)

Observabilité Révisions Source Déclencheurs Réseau Sécurité YAML

Métriques

Journaux

Niveau de gravité

Par défaut



Filtrer

Rechercher dans tous les champs et valeurs



Journaux

Niveau de gravité

Horodatage

Résumé

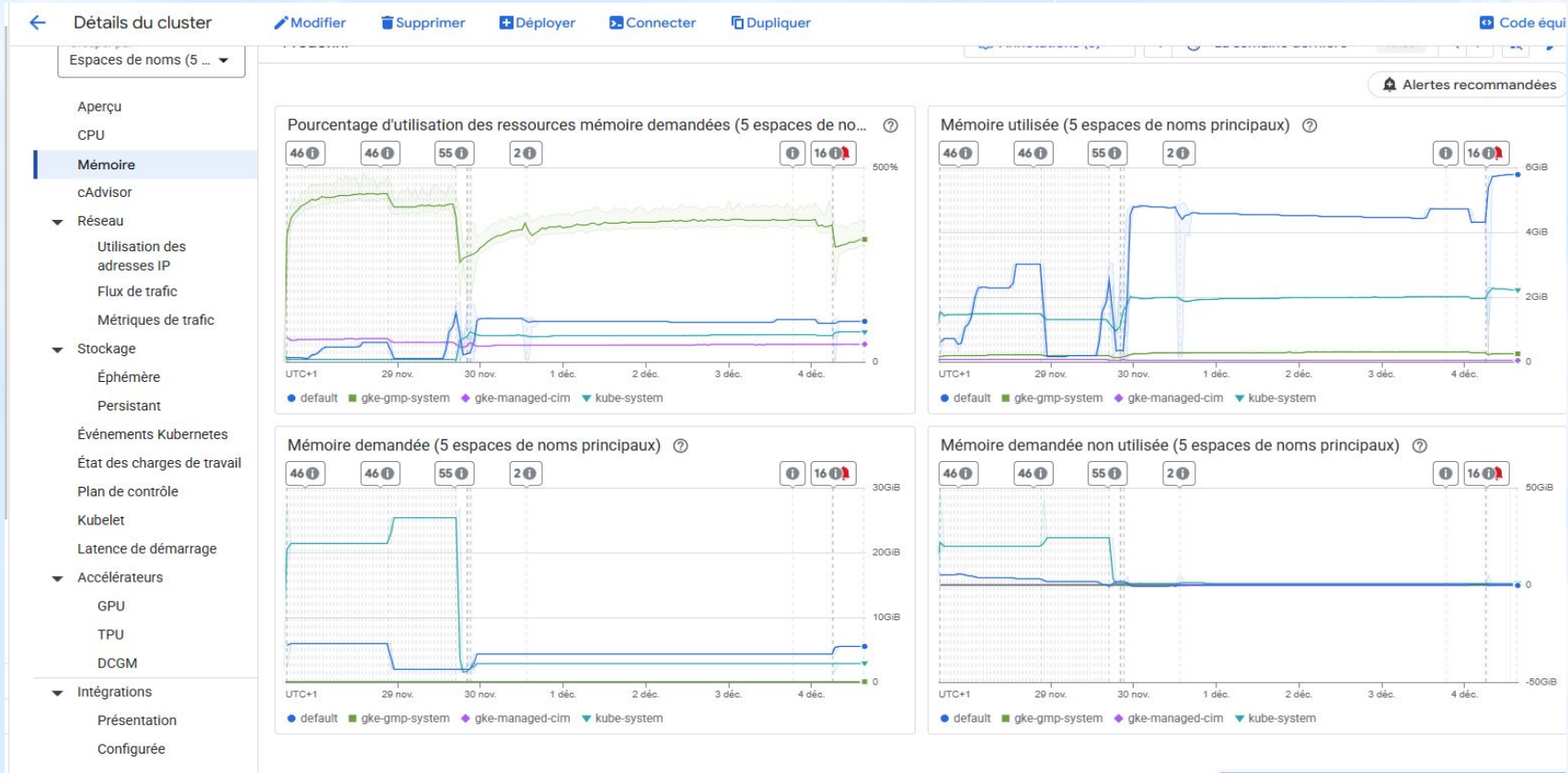
SLO

Erreurs

>	*	2025-12-04 15:00:03.346	HNEC	[INFO]	2025-12-04 14:00:03 - Fetching CSV data from FastAPI at https://dct-00-airlines-api-901450845940.europe-west9.run.app/update_scheduled_d1/export?format=10...
>	*	2025-12-04 15:00:03.346	HNEC	[INFO]	2025-12-04 14:00:03 - Reading CSV data into DataFrame...
>	*	2025-12-04 15:00:03.352	HNEC	[INFO]	2025-12-04 14:00:03 - Renaming DataFrame columns...
>	*	2025-12-04 15:00:03.353	HNEC	[INFO]	2025-12-04 14:00:03 - Columns renamed successfully.
>	*	2025-12-04 15:00:03.353	HNEC	[INFO]	2025-12-04 14:00:03 - Preparing data for COPY...
>	*	2025-12-04 15:00:03.354	HNEC	[INFO]	2025-12-04 14:00:03 - Connecting to PostgreSQL...
>	*	2025-12-04 15:00:03.371	HNEC	[INFO]	2025-12-04 14:00:03 - Connection established.
>	*	2025-12-04 15:00:03.371	HNEC	[INFO]	2025-12-04 14:00:03 - Inserting data into PostgreSQL table 'mongodb_future_d1' ...
>	*	2025-12-04 15:00:03.379	HNEC	[INFO]	2025-12-04 14:00:03 - Data inserted successfully.
>	*	2025-12-04 15:00:03.405	HNEC	[INFO]	2025-12-04 14:00:03 - Running SQL string...
>	*	2025-12-04 15:00:03.405	HNEC	[INFO]	2025-12-04 14:00:03 - Executing: ALTER TABLE mongodb_future_d1 ADD COLUMN flight_id VARCHAR(50)...
>	*	2025-12-04 15:00:03.412	HNEC	[ERROR]	2025-12-04 14:00:03 - SQL execution failed: column "flight_id" of relation "mongodb_future_d1" already exists
>	*	2025-12-04 15:00:03.412	HNEC	[INFO]	2025-12-04 14:00:03 - Executing: UPDATE mongodb_future_d1 SET flight_id = CONCAT (id, flightLegs_depInfo_airport...
>	*	2025-12-04 15:00:03.415	HNEC	[ERROR]	2025-12-04 14:00:03 - SQL execution failed: current transaction is aborted, commands ignored until end of transaction block
>	*	2025-12-04 15:00:03.418	HNEC	[INFO]	2025-12-04 14:00:03 - SQL execution finished.
>	*	2025-12-04 15:00:03.440	HNEC	[INFO]	2025-12-04 14:00:03 - Running SQL string...
>	*	2025-12-04 15:00:03.440	HNEC	[INFO]	2025-12-04 14:00:03 - Executing: drop index if exists mongodb_future_d1_flight_id...
>	*	2025-12-04 15:00:03.449	HNEC	[INFO]	2025-12-04 14:00:03 - SQL execution successful.
>	*	2025-12-04 15:00:03.450	HNEC	[INFO]	2025-12-04 14:00:03 - Executing: CREATE INDEX mongodb_future_d1_flight_id ON mongodb_future_d1(flight_id)...
>	*	2025-12-04 15:00:03.459	HNEC	[INFO]	2025-12-04 14:00:03 - SQL execution successful.
>	*	2025-12-04 15:00:03.464	HNEC	[INFO]	2025-12-04 14:00:03 - SQL execution finished.

Cloud Monitoring

Journaux de métriques



Conclusion

1. Collecte des données
2. Stockage et tri des données - mongoDB
3. Base de données relationnelle - PostgreSQL
4. Exploration des données
5. Apprentissage machine
6. Conteneurisation, APIs et dashboards
7. Déploiement sur le cloud



Pistes d'amélioration

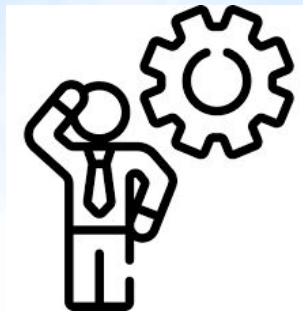
Projet Airlines

Tests unitaires

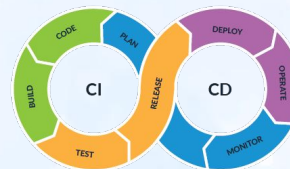
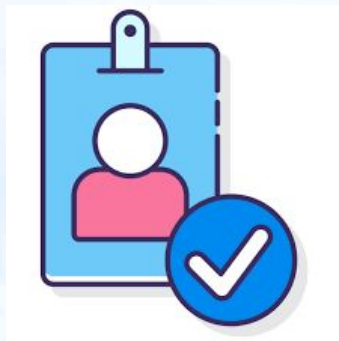
Tests
d'intégration

Optimisation des
modèles ML

K-Means



Apache
Airflow



Google Cloud Platform

